

Analyse Technique : Système de Verrouillage Avancé React Native

Ce document présente une analyse architecturale et technique détaillée de l'application React Native (`MonProjetNative`), qui implémente un système de sécurité et d'authentification biométrique/gestuelle robuste.

1. Stack Technologique et Dépendances

L'application repose sur un socle moderne, assurant performance et typage strict.

- **Framework Principal :** `React Native (0.85.3)` - Développement multiplateforme (iOS/Android).
- **Langage :** `TypeScript` - Apporte un typage statique fort, essentiel pour éviter les erreurs d'exécution lors de la manipulation des objets natifs et des capteurs.
- **UI et Layout :** Utilisation des composants de base (`SafeAreaView`, `View`, `Text`, `TouchableOpacity`) couplés à `StyleSheet` et `react-native-safe-area-context` pour une intégration parfaite sur les écrans à encoche (notches/dynamic islands).

Bibliothèques Matérielles Essentielles (Native Modules)

1. `react-native-biometrics` (v3.0.1)

- **Rôle :** Interface avec les API biométriques du système d'exploitation (Face ID, Touch ID, Fingerprint).
- **Mécanisme :** Permet la détection du matériel disponible via `isSensorAvailable()` et l'invocation du prompt d'authentification sécurisé via `simplePrompt()`.

2. `react-native-sensors` (v7.3.6) & `rxjs` (v7.8.2)

- **Rôle :** Extraction des données de l'accéléromètre en temps réel pour implémenter la fonctionnalité "Shake to Unlock" (Secouer pour déverrouiller).
- **Mécanisme :** `rxjs` est utilisé pour traiter le flux continu de données capteur (Reactive Programming), appliquant des filtres (`filter`) et des transformations mathématiques (`map`) de manière extrêmement performante.

3. `react-native-sound` (v0.13.0)

- **Rôle :** Lecture de fichiers audio locaux depuis le bundle natif (sons système : déverrouillage, erreur, alarme).

2. Architecture Logicielle et Gestion d'État

Le composant principal (`App.tsx`) est construit autour des **Hooks React**, gérant un cycle de vie complexe pour l'écran de verrouillage. L'architecture démontre une excellente maîtrise des problématiques de concurrence et de gestion mémoire.

Stratégie d'utilisation des Hooks

- **useState** : Maintient l'état réactif de l'UI.
 - *États de sécurité* : `isLocked`, `failedAttempts`, `lockoutUntil`, `secondsRemaining`.
 - *États interactifs* : `isShakeMode`, `shakeCount`, `sensorType`.
- **useEffect** : Contrôle scrupuleusement les effets de bord (side-effects).
 - **Gestion Mémoire** : Chargement des instances audio au montage (`soundUnlock`, `soundError`, `soundAlarm`) et libération explicite (`.release()`) au démontage pour prévenir les fuites de mémoire (memory leaks).
 - **Timers Asynchrones** : Mise en place et nettoyage (`clearInterval`) du minuteur de pénalité de 60 secondes.
 - **Abonnements** : Souscription et désinscription propre aux événements de l'accéléromètre.
- **useRef (Résolution de problèmes avancés)** :
 - **Anti-Stale Closures** : `lockoutUntilRef` est utilisé pour permettre au `setInterval` de lire la valeur de blocage actuelle sans être victime du piège des closures obsolètes en JavaScript.
 - **Verrous d'exclusion mutuelle (Mutex)** : `isErrorSoundPlaying` empêche l'exécution simultanée (superposition) de plusieurs sons d'erreur si l'utilisateur tapote frénétiquement ou échoue rapidement.



3. Fonctionnalités Métier et Implémentation Détaillée

A. Authentification Biométrique Avancée

L'implémentation va au-delà d'un simple appel d'API. Elle inclut une gestion d'erreurs granulaires :

- L'état `isBiometricPending` agit comme une barrière empêchant de multiples appels simultanés au capteur (anti-spam).
- **Distinction des erreurs** : Le code différencie intelligemment un "échec d'empreinte" (l'utilisateur s'est trompé) d'une "erreur système" (ex. le capteur n'est pas disponible ou aucune empreinte n'est configurée). Seuls les véritables échecs incrémentent le compteur de pénalité, offrant ainsi une UX juste.

B. Algorithme de Détection de Secousse (Shake-to-Unlock)

Ce mode de secours utilise une approche algorithmique sophistiquée :

1. **Traitement de Signal (RxJS)** : La norme du vecteur d'accélération tridimensionnel est calculée : $\text{Force} = \sqrt{x^2 + y^2 + z^2}$.
2. **Filtrage** : Seules les forces dépassant le seuil `SHAKE_THRESHOLD` (22 m/s²) sont conservées.
3. **Anti-rebond (Debounce)** : Un délai de `SHAKE_TIMEOUT` (800ms) vérifié par timestamp (`Date.now() - lastShakeTime.current`) empêche de comptabiliser un seul mouvement ample comme plusieurs secousses distinctes.
4. **Règle Métier Dynamique** : Le nombre de secousses requises n'est pas statique. Il est calculé via l'équation modulo `(jour2 mod 5)`. Le vendredi (jour 5) contourne le système pour un accès libre.

C. Système Anti-Brute-Force (Lockout)

Une sécurité robuste bloque l'application après un nombre défini de tentatives infructueuses :

- Dès que `failedAttempts` atteint `MAX_ATTEMPTS` (4), une variable timestamp `lockoutUntil` est fixée à `Date.now() + 60000` (1 minute dans le futur).
- Une alarme sonore (`alarm.mp3`) est déclenchée.
- L'interface utilisateur désactive totalement le bouton biométrique et le mode secousse, remplaçant les contrôles par un compte à rebours dynamique (`secondsRemaining`).
- L'abandon volontaire du mode "Secousse" (bouton retour après avoir commencé) est intelligemment comptabilisé comme un échec pour éviter de réinitialiser les compteurs en abusant de cette fonction.

Conclusion

L'architecture de l'application témoigne d'une approche mature et sécurisée du développement mobile. L'isolation des états, la gestion méticuleuse de la mémoire via les Hooks, et la robustesse des traitements asynchrones (capteurs, biométrie, minuteurs) en font une implémentation fiable d'un système de verrouillage natif.